Bodegas

Pepe Calderón, Fernando Sola, Daniel Ayala, Inma Hernández, Margarita Cruz, Carlos Arévalo, David Ruiz

Contenido

1. Requisitos . . . 3
2. Requisitos de información (RI) . . . 3
   2.1. RI-1: Bodega . . . 3
   2.2. RI-2: Vino . . . 3
   2.3. RI-3: Uvas . . . 3
   2.4. RI-4: Cosechas . . . 3
3. Reglas de negocio (RN) . . . 3
   3.1. RN-1: Unicidad . . . 3
   3.2. RN-2: Tiempo de crianza . . . 3
   3.3. RN-3: Graduación alcohólica . . . 4
   3.4. RN-4: Cosechas . . . 4
4. Requisitos funcionales (RF) . . . 4
   4.1. RF-1: Informes simples de Bodegas . . . 4
5. Pruebas de aceptación (PA) . . . 4
   5.1. PA-1: Bodegas . . . 4
   5.2. PA-2: Vinos . . . 4
6. Modelo Conceptual . . . 5
7. Diagrama de clases . . . 5
8. Diagrama de objetos . . . 5
9. Posible extensión . . . 6
10. Modelo Relacional (v1) . . . 6
11. Versión 1: Una relación para cada clase de la jerarquía . . . 6
   11.1. Intensión . . . 6
   11.2. Extensión . . . 7
   11.3. Álgebra relacional . . . 8
12. Versión 2: Una relación para cada subclase . . . 9
   12.1. Intensión . . . 9
   12.2. Extensión . . . 10
13. Modelo Relacional (v2) . . . 10
14. Versión 2: Una relación para cada subclase . . . 10
   14.1. Intensión . . . 10
   14.2. Extensión . . . 11
15. Modelo Tecnológico (v1) . . . 11
16. Script SQL para crear la base de datos . . . 12
17. Script SQL para la carga inicial de datos . . . 13
18. Consultas . . . 16
19. Modelo Tecnológico (v2) . . . 17
20. Script SQL para crear la base de datos . . . 17

21. Script SQL para la carga inicial de datos 19
22. Consultas 22
23. SQL Avanzado 23
24. Pruebas SQL 24
25. Pruebas HTTP 33

Versión PDF disponible

Bodegas

1. Requisitos

Catálogo de Requisitos

2. Requisitos de información (RI)

2.1. RI-1: Bodega

- Como: Profesor de la asignatura
- Quiero: Poder almacenar la siguiente información de las bodegas: nombre y denominación de origen.
- Para: Que el estudiante realice a partir de este requisito el modelo conceptual, relacional y tecnológico.

2.2. RI-2: Vino

- Como: Profesor de la asignatura
- Quiero: Poder almacenar la siguiente información de los vinos: nombre, grados de alcohol y Uvas que lo componen. Los vinos pueden ser de Jóvenes o Crianzas, en función al tiempo que pasen en barrica y botella. Además, para los vinos Crianzas, se debe almacenar las cosechas.
- Para: Que el estudiante realice a partir de este requisito el modelo conceptual, relacional y tecnológico.

2.3. RI-3: Uvas

- Como: Profesor de la asignatura
- Quiero: Poder almacenar la siguiente información de las uvas: nombre (Tempranillo, Garnacha, Carbernet Sauvignon, …)
- Para: Que el estudiante realice a partir de este requisito el modelo conceptual, relacional y tecnológico.

2.4. RI-4: Cosechas

- Como: Profesor de la asignatura
- Quiero: Poder almacenar la siguiente información de las cosechas: año y calidad (Excelente, Muy buena, Buena, Normal, Mala)
- Para: Que el estudiante realice a partir de este requisito el modelo conceptual, relacional y tecnológico.

3. Reglas de negocio (RN)

3.1. RN-1: Unicidad

- Como: Profesor de la asignatura
- Quiero: Que no se repitan el nombre de las bodegas, vinos o uvas en el sistema
- Para: Que el estudiante practique con restricciones simples.

3.2. RN-2: Tiempo de crianza

- Como: Profesor de la asignatura

- Quiero: Que el tiempo de crianza de un vino Crianza sea de al menos 24 meses, de los cuales entre 6 y 12 meses son en barrica y el resto en botella. El vino joven generalmente no pasa tiempo en barrica, o si lo hace, es muy breve (menos de 6 meses)
- Para: Que el estudiante practique con restricciones simples.

3.3. RN-3: Graduación alcohólica

- Como: Profesor de la asignatura
- Quiero: Que la graduación alcohólica de un vino suele estar entre los 10 y 15 grados
- Para: Que el estudiante practique con restricciones complejas.

3.4. RN-4: Cosechas

- Como: Profesor de la asignatura
- Quiero: Cada vino de crianza tiene como máximo una cosecha por año, y tiene al menos una cosecha
- Para: Que el estudiante practique con restricciones complejas.

4. Requisitos funcionales (RF)

4.1. RF-1: Informes simples de Bodegas

- Como: Profesor de la asignatura
- Quiero: Seleccionar todas las bodegas con denominación de origen Rioja. Listado de vinos con sus uvas. Crianazas con sus cosechas. Mostrar todas las bodegas que producen vinos tanto jóvenes como crianzas. Nombre de las bodegas y vinos que están compuestos, al menos, con uva 'Tempranillo'. Total de crianzas por cosecha. Nombre del vino joven con más grados
- Para: Que el estudiante practique con consultas simples.

5. Pruebas de aceptación (PA)

5.1. PA-1: Bodegas

1. ✅ Crear una nueva bodega con todos los datos correctos según las reglas de negocio
2. ❌ Crear una nueva bodega sin nombre.
3. ❌ Crear una nueva bodega con el nombre repetido.
4. ❌ Crear una nueva bodega sin denominación de origen.

5.2. PA-2: Vinos

1. ✅ Crear un nuevo vino Joven con todos los datos correctos según las reglas de negocio
2. ✅ Crear un nuevo vino Crianza con todos los datos correctos según las reglas de negocio
3. ❌ Crear un nuevo vino Joven sin nombre.
4. ❌ Crear un nuevo vino Crianza sin nombre.
5. ❌ Crear un nuevo vino Joven con el nombre repetido.
6. ❌ Crear un nuevo vino Crianza con el nombre repetido.
7. ❌ Crear un nuevo vino Joven sin grados de alcohol.
8. ❌ Crear un nuevo vino Crianza sin grados de alcohol.

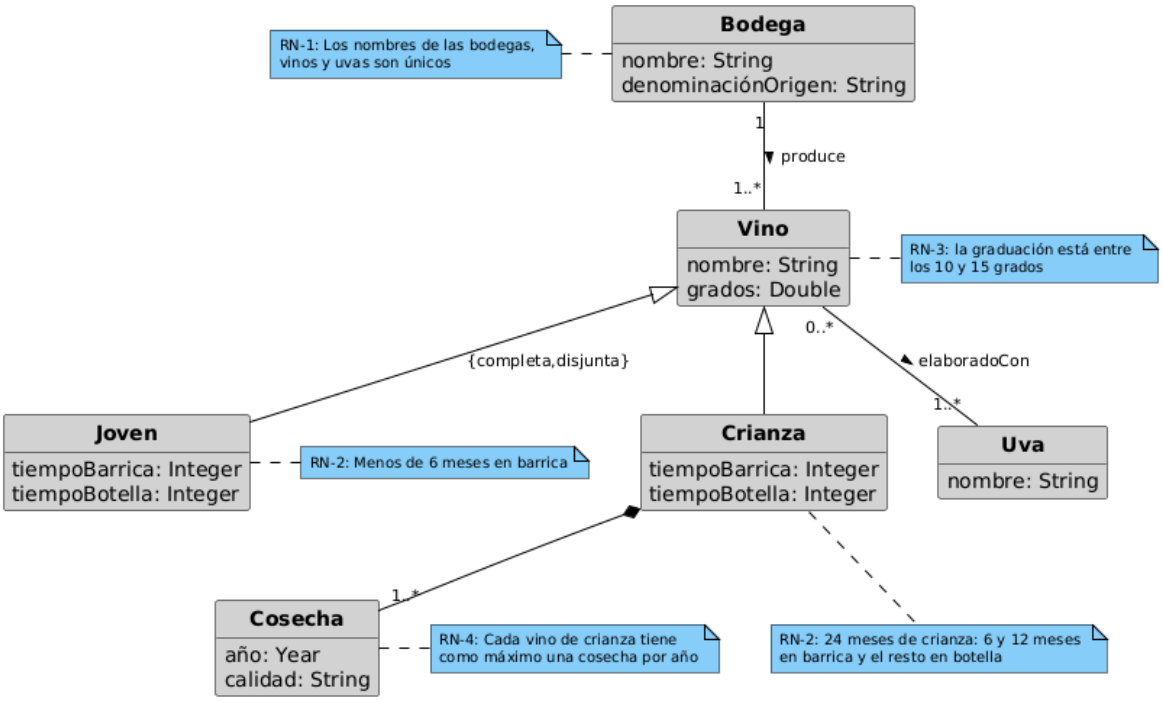
9. ❌ Crear un nuevo vino Joven con graduación incorrecta.
10. ❌ Crear un nuevo vino Crianza con graduación incorrecta.
11. ❌ Crear un nuevo vino Joven sin bodega.
12. ❌ Crear un nuevo vino Crianza sin bodega.

6. Modelo Conceptual

Modelo conceptual

7. Diagrama de clases

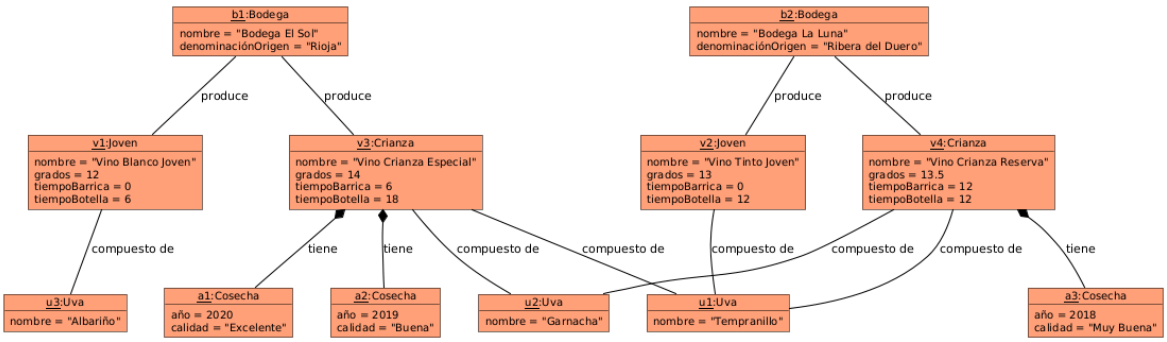
- Producción: cada Bodega produce 1..* Vino; cada Vino pertenece a 1 Bodega.
- Generalización: Vino se especializa en Joven y Crianza con partición {completa, disjunta}.
- Crianza–Cosecha: composición 1..* que modela las cosechas de un vino de crianza.
- Vino–Uva: relación M:N (elaboradoCon) entre Vino y Uva: un vino puede mezclar varias uvas y una uva intervenir en múltiples vinos.



8. Diagrama de objetos

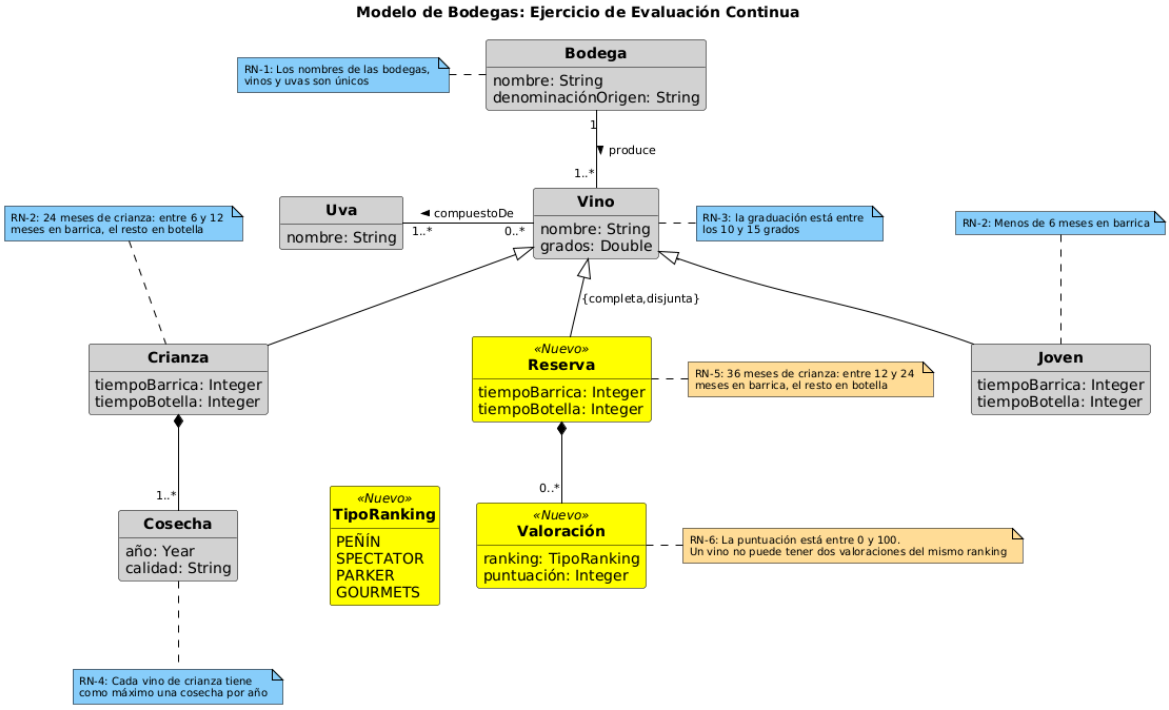
Las instancias muestran dos bodegas (b1, b2) y cuatro vinos: j1, j2 (jóvenes) y c1, c2 (crianza), con sus cosechas y uvas.

- Producción: b1 produce j1 y c1; b2 produce j2 y c2.
- Composición de crianza: c1 tiene cosechas a1 y a2; c2 tiene a3 (respeta 1..*).
- Mezclas de uva: j1 → u3; j2 → u1; c1 → u1+u2; c2 → u1+u2, ilustrando la M:N Vino–Uva.
- El diagrama de objetos ejemplifica las cardinalidades y la especialización sin violar las RN indicadas en el diagrama de clases.



9. Posible extensión

- Jerarquía extendida: Vino se especializa en Joven, Crianza y Reserva {completa, disjunta}.
- Reserva compone 0..* Valoración (ranking: TipoRanking, puntuación 0..100); no puede haber dos valoraciones del mismo ranking para un mismo vino (RN-6).
- Se mantienen Bodega–Vino (1..*), Crianza–Cosecha (1..*), y Vino–Uva (M:N) como en el modelo base.
- RN-5: Reserva 36 meses (12–24 en barrica; resto en botella). Además, se mantienen RN-1..RN-4.



10. Modelo Relacional (v1)

Modelo relacional

11. Versión 1: Una relación para cada clase de la jerarquía

11.1. Intensión

```
Bodegas(bodegaId, nombre, denominaciónOrigen)
    PK(bodegaId)
```

```
    AK(nombre)
Vinos(vinoId, bodegaId, nombre, grados)
    PK(vinoId)
    FK(bodegaId) / Bodegas
    AK(nombre)
Jóvenes(vinoId, tiempoBarrica, tiempoBotella)
    PK(vinoId)
    FK(vinoId) / Vinos
Crianzas(vinoId, tiempoBarrica, tiempoBotella)
    PK(vinoId)
    FK(vinoId) / Vinos
Uvas(uvaId, nombre)
    PK(uvaId)
    AK(nombre)
Cosechas(cosechaId, crianzaId, año, calidad)
    PK(cosechaId)
    FK(crianzaId) / Crianzas
    AK(cosechaId, crianzaId, año)
VinosUvas(vinoUvaId, vinoId, uvaId)
    PK(vinoUvaId)
    FK(vinoId) / Vinos
    FK(uvaId) / Uvas
    AK(vinoId, uvaId)
```

11.2. Extensión

```
Bodegas = {
    (b1, "Bodegas El Sol", "Rioja"),
    (b2, "Bodegas La Luna", "Ribera del Duero")
}
Vinos = {
    (v1, b1, "Vino Blanco Joven", 12),
    (v2, b2, "Vino Tinto Joven", 13),
    (v3, b1, "Vino Crianza Especial", 14),
    (v4, b2, "Vino Crianza Reserva", 13.5)
}
Jóvenes = { (v1, 0, 6), (v2, 0, 12) }
Crianzas = { (v3, 6, 18), (v4, 12, 12) }
Uvas = {
    (u1, "Tempranillo"),
    (u2, "Garnacha"),
    (u3, "Albarino")
}
Cosechas = {
    (c1, v3, 2020, "Excelente"),
    (c2, v3, 2019, "Buena"),
    (c3, v4, 2018, "Muy buena")
}
VinosUvas = {
    (vu1, v1, u3),
    (vu2, v2, u1),
    (vu3, v3, u2),
    (vu4, v3, u1),
    (vu5, v4, u2),
    (vu6, v4, u1)
}
```

11.3. Álgebra relacional

-Renombrado:

$$B = \underset{B(bid,nb,do)}{\rho}(Bodegas)$$

$$V = \underset{V(vid,bid,nv,g)}{\rho}(Vinos)$$

$$J = \underset{J(vid,tba,tbo)}{\rho}(Jóvenes)$$

$$C = \underset{C(vid,tba,tbo)}{\rho}(Crianzas)$$

$$U = \underset{U(uid,nu)}{\rho}(Uvas)$$

$$Co = \underset{Co(coid,vid,a,c)}{\rho}(Cosechas)$$

$$VU = \underset{VU(vuid,vid,uid)}{\rho}(VinosUvas)$$

- Seleccionar todas las bodegas con denominación de origen Rioja:

$$Riojas \leftarrow \underset{do=\text{'Rioja'}}{\sigma}(B)$$

- Listado de vinos con sus uvas:

$$VVUU \leftarrow V \bowtie VU \bowtie U$$

La intensión de la relación derivada VVUU sería la unión de los conjuntos de atributos de las relaciones V, VU y U:

```
Intensión(VVUU) = {vid, bid, nv, g, vuid, uid, nu}
```

La extensión de VVUU quedaría con las siguientes tuplas:

```
Extensión(VVUU) = {
    (1, 1, "Vino Blanco Joven", 12, 1, 3, "Albarino"),
    (2, 2, "Vino Tinto Joven", 13, 2, 1, "Tempranillo"),
    (3, 1, "Vino Crianza Especial", 14, 4, 2, "Garnacha"),
    (3, 1, "Vino Crianza Especial", 14, 5, 1, "Tempranillo"),
    (4, 2, "Vino Crianza Reserva", 13.5, 6, 2, "Garnacha"),
    (4, 2, "Vino Crianza Reserva", 13.5, 7, 1, "Tempranillo")
}
```

- Crianzas con sus cosechas:

$$CCo \leftarrow V \bowtie Co$$

En este caso, la intensión de la relación derivada CCo sería la unión de los conjuntos de atributos de las relaciones C y Co, y la extensión tendría las siguientes tuplas:

```
Intensión(CCo) = {vid, bid, nv, g, coid, a, c}

Extensión(CCo) = {
    (3, 1, "Vino Crianza Especial", 14, 1, 2020, "Excelente"),
    (3, 1, "Vino Crianza Especial", 14, 2, 2019, "Buena"),
```

```
    (4, 2, "Vino Crianza Reserva", 13.5, 3, 2018, "Muy buena")
}
```

- Mostrar todas las bodegas que producen vinos tanto jóvenes como crianzas:

$$BV \leftarrow B \bowtie V$$

- Nombre de las bodegas y vinos que están compuestos, al menos, con uva "Tempranillo":

$$BT \leftarrow \prod_{nb,nv}\left(\sigma_{nu=\text{'Tempranillo'}}(BV \bowtie VU \bowtie U)\right)$$

- Total de cosechas por vino de crianza:

$$TotalCrianzas \leftarrow {}^{\text{COUNT}(*)}_{\gamma}\, coid(C \bowtie Co)$$

- Nombre del vino joven con más grados:

$$max \leftarrow {}^{\text{MAX}(g)}_{\gamma}(V \bowtie J)$$

$$VinoMasGrados \leftarrow \prod_{nv}\left(\sigma_{g=max}(V \bowtie J)\right)$$

- Número de vinos crianza por cosecha:

$$NumCCo \leftarrow {}^{\text{COUNT}(vid)}_{\gamma}\, coid(Co)$$

- Bodegas con más vinos:

$$BodegasMasVinos \leftarrow {}^{\text{MAX}(n)}_{\gamma}\left(\rho_{n \leftarrow \text{COUNT}(vid)}\left({}^{\text{COUNT}(vid)}_{\gamma}\, bid(BV)\right)\right)$$

- Vinos que tienen, al menos, las mismas uvas que el vino 'v1':

$$VinosUvasV1 \leftarrow \prod_{vid,nv}\left(\frac{\prod_{vid,uid}(VU)}{\prod_{uid}\left(\sigma_{vid=v1}(VU)\right)} \bowtie V\right)$$

12. Versión 2: Una relación para cada subclase

12.1. Intensión

```
Bodegas(bodegaId, nombre, denominaciónOrigen)
    PK(bodegaId)
    AK(nombre)
Jóvenes(jovenId, bodegaId, nombre, grados, tiempoBarrica, tiempoBotella)
    PK(jovenId)
    FK(bodegaId) / Bodegas
    AK(nombre)
Crianzas(crianzaId, bodegaId, nombre, grados, tiempoBarrica, tiempoBotella)
    PK(crianzaId)
    FK(bodegaId) / Bodegas
    AK(nombre)
Uvas(uvaId, nombre)
    PK(uvaId)
    AK(nombre)
Cosechas(cosechaId, crianzaId, año, calidad)
    PK(cosechaId)
```

```
    FK(crianzaId) / Crianzas
    AK(cosechaId, crianzaId, año)
VinosUvas(vinoUvaId, jovenId*, crianzaId*, uvaId)
    PK(vinoUvaId)
    FK(jovenId) / Jóvenes
    FK(crianzaId) / Crianzas
    FK(uvaId) / Uvas
    AK(jovenId, uvaId)
    AK(crianzaId, uvaId)
    * Restricción: jovenId y crianzaId deben ser disjuntos.
```

12.2. Extensión

```
Bodegas = {
    (b1, "Bodegas El Sol", "Rioja"),
    (b2, "Bodegas La Luna", "Ribera del Duero")
}
Jóvenes = {
    (j1, b1, "Vino Blanco Joven", 12, 6, 0),
    (j2, b2, "Vino Tinto Joven", 13, 0, 12),
}
Crianzas = {
    (c1, b1, "Vino Crianza Especial", 14, 6, 18),
    (c2, b2, "Vino Crianza Reserva", 13.5, 12, 12)
}
Uvas = {
    (u1, "Tempranillo"),
    (u2, "Garnacha"),
    (u3, "Albarino")
}
Cosechas = {
    (co1, c1, 2020, "Excelente"),
    (co2, c1, 2019, "Buena"),
    (co3, c2, 2018, "Muy buena")
}
VinosUvas = {
    (vu1, j1, null, u3),
    (vu2, j2, null, u1),
    (vu3, null, c1, u2),
    (vu4, null, c1, u1),
    (vu5, null, c2, u2),
    (vu6, null, c2, u1)
}
```

13. Modelo Relacional (v2)

Modelo relacional

14. Versión 2: Una relación para cada subclase

14.1. Intensión

```
Bodegas(bodegaId, nombre, denominaciónOrigen)
    PK(bodegaId)
    AK(nombre)
Jóvenes(jovenId, bodegaId, nombre, grados, tiempoBarrica, tiempoBotella)
    PK(jovenId)
    FK(bodegaId) / Bodegas
```

```
    AK(nombre)
Crianzas(crianzaId, bodegaId, nombre, grados, tiempoBarrica, tiempoBotella)
    PK(crianzaId)
    FK(bodegaId) / Bodegas
    AK(nombre)
Uvas(uvaId, nombre)
    PK(uvaId)
    AK(nombre)
Cosechas(cosechaId, crianzaId, año, calidad)
    PK(cosechaId)
    FK(crianzaId) / Crianzas
    AK(cosechaId, crianzaId, año)
VinosUvas(vinoUvaId, jovenId*, crianzaId*, uvaId)
    PK(vinoUvaId)
    FK(jovenId) / Jóvenes
    FK(crianzaId) / Crianzas
    FK(uvaId) / Uvas
    AK(jovenId, uvaId)
    AK(crianzaId, uvaId)
    * Restricción: jovenId y crianzaId deben ser disjuntos.
```

14.2. Extensión

```
Bodegas = {
    (b1, "Bodegas El Sol", "Rioja"),
    (b2, "Bodegas La Luna", "Ribera del Duero")
}
Jóvenes = {
    (j1, b1, "Vino Blanco Joven", 12, 6, 0),
    (j2, b2, "Vino Tinto Joven", 13, 0, 12),
}
Crianzas = {
    (c1, b1, "Vino Crianza Especial", 14, 6, 18),
    (c2, b2, "Vino Crianza Reserva", 13.5, 12, 12)
}
Uvas = {
    (u1, "Tempranillo"),
    (u2, "Garnacha"),
    (u3, "Albarino")
}
Cosechas = {
    (co1, c1, 2020, "Excelente"),
    (co2, c1, 2019, "Buena"),
    (co3, c2, 2018, "Muy buena")
}
VinosUvas = {
    (vu1, j1, null, u3),
    (vu2, j2, null, u1),
    (vu3, null, c1, u2),
    (vu4, null, c1, u1),
    (vu5, null, c2, u2),
    (vu6, null, c2, u1)
}
```

15. Modelo Tecnológico (v1)

Modelo tecnológico. Versión 1: Una relación para cada clase de la jerarquía

16. Script SQL para crear la base de datos

Bodegas/createDB.sql

```sql
--
-- Autor: David Ruiz
-- Fecha: Octubre de 2024
-- Descripción: Script para crear la BD del ejercicio de Bodegas
--

DROP DATABASE IF EXISTS BodegasDB;
CREATE DATABASE BodegasDB;
USE BodegasDB;

DELIMITER //
CREATE OR REPLACE PROCEDURE p_create_db()
BEGIN
  CREATE OR REPLACE TABLE bodegas (
     bodega_id INT NOT NULL AUTO_INCREMENT,
     nombre VARCHAR(255) NOT NULL,
     denominacion_origen VARCHAR(255) NOT NULL,
    PRIMARY KEY (bodega_id),
    CONSTRAINT RN_1_Unicidad UNIQUE (nombre)
  );

  CREATE OR REPLACE TABLE vinos (
     vino_id INT NOT NULL AUTO_INCREMENT,
     bodega_id INT NOT NULL,
     nombre VARCHAR(255) NOT NULL,
     grados DECIMAL(5, 2) NOT NULL,
    PRIMARY KEY (vino_id),
    UNIQUE (nombre),
     FOREIGN KEY (bodega_id) REFERENCES bodegas(bodega_id),
    CONSTRAINT RN_3_Grados
      CHECK (grados >= 10 AND grados <= 15)
  );

  CREATE OR REPLACE TABLE jovenes (
     vino_id INT NOT NULL,
    tiempo_barrica INT NOT NULL,
    tiempo_botella INT NOT NULL,
    PRIMARY KEY (vino_id),
     FOREIGN KEY (vino_id) REFERENCES vinos(vino_id),
    CONSTRAINT RN_2_TiempoCrianza
      CHECK (tiempo_barrica <=6 AND tiempo_barrica + tiempo_botella <=12 )
  );

  CREATE OR REPLACE TABLE crianzas (
     vino_id INT NOT NULL,
    tiempo_barrica INT NOT NULL,
    tiempo_botella INT NOT NULL,
    PRIMARY KEY (vino_id),
     FOREIGN KEY (vino_id) REFERENCES vinos(vino_id),
    CONSTRAINT RN_2_TiempoCrianza
      CHECK (tiempo_barrica >=6 AND tiempo_barrica + tiempo_botella >=24 )
  );
```

```
  CREATE OR REPLACE TABLE uvas (
     uva_id INT NOT NULL AUTO_INCREMENT,
     nombre VARCHAR(255) NOT NULL,
    PRIMARY KEY (uva_id),
     CONSTRAINT RN_1_Unicidad UNIQUE (nombre)
  );

  CREATE OR REPLACE TABLE cosechas (
     cosecha_id INT AUTO_INCREMENT ,
     vino_id INT NOT NULL,
     año YEAR NOT NULL,
     calidad VARCHAR(255) NOT NULL,
    PRIMARY KEY (cosecha_id),
     FOREIGN KEY (vino_id) REFERENCES vinos(vino_id)
      ON DELETE CASCADE,
    UNIQUE (cosecha_id, año)
  );

  CREATE OR REPLACE TABLE vinos_uvas (
      vino_uva_id INT AUTO_INCREMENT PRIMARY KEY,
      vino_id INT NOT NULL,
      uva_id INT NOT NULL,
      UNIQUE (vino_id, uva_id),
      FOREIGN KEY (vino_id) REFERENCES vinos(vino_id),
      FOREIGN KEY (uva_id) REFERENCES uvas(uva_id)
  );
END;
//
DELIMITER ;

CALL p_create_db();
```

17. Script SQL para la carga inicial de datos

Bodegas/populateDB.sql

```
--
-- Autor: David Ruiz
-- Fecha: Octubre de 2024
-- Descripción: Script para poblar la BD del ejercicio de Bodegas
--
```

```sql
delimiter //
CREATE OR REPLACE PROCEDURE p_populate_db()
BEGIN
  -- Vaciar tablas
  DELETE FROM vinos_uvas;
  DELETE FROM cosechas;
  DELETE FROM jovenes;
  DELETE FROM crianzas;
  DELETE FROM vinos;
  DELETE FROM uvas;
  DELETE FROM bodegas;
  -- Insertar datos en Bodegas
  INSERT INTO bodegas (bodega_id, nombre, denominacion_origen) VALUES
  (1, 'Bodegas El Sol', 'Rioja'),
  (2, 'Bodegas La Luna', 'Ribera del Duero');

  -- Insertar datos en Vinos
  INSERT INTO vinos (vino_id, bodega_id, nombre, grados) VALUES
  (1, 1, 'Vino Blanco Joven', 12),
  (2, 2, 'Vino Tinto Joven', 13),
  (3, 1, 'Vino Crianza Especial', 14),
  (4, 2, 'Vino Crianza Reserva', 13.5);

  -- Insertar datos en Jóvenes
  INSERT INTO jovenes (vino_id, tiempo_barrica, tiempo_botella) VALUES
  (1, 0, 6),
  (2, 0, 12);

  -- Insertar datos en Crianzas
  INSERT INTO crianzas (vino_id, tiempo_barrica, tiempo_botella) VALUES
  (3, 6, 18),
  (4, 12, 12);

  -- Insertar datos en Uvas
  INSERT INTO uvas (uva_id, nombre) VALUES
  (1, 'Tempranillo'),
  (2, 'Garnacha'),
  (3, 'Albarino');

  -- Insertar datos en Cosechas
  INSERT INTO cosechas (cosecha_id, vino_id, año, calidad) VALUES
  (1, 3, 2020, 'Excelente'),
  (2, 3, 2019, 'Buena'),
  (3, 4, 2018, 'Muy buena');

  -- Insertar datos en VinosUvas
  INSERT INTO vinos_uvas (vino_uva_id, vino_id, uva_id) VALUES
  (1, 1, 3),
  (2, 2, 1),
  (3, 3, 2),
  (4, 3, 1),
  (5, 4, 2),
  (6, 4, 1);
/*
  -- Insertar datos adicionales en Vinos
```

```
INSERT INTO Vinos (vinoId, bodegaId, nombre, grados) VALUES
(5, 1, 'Vino Blanco Reserva', 12.5),
(6, 2, 'Vino Tinto Gran Reserva', 14),
(7, 1, 'Vino Rosado Joven', 11.5),
(8, 2, 'Vino Blanco Crianza', 13),
(9, 1, 'Vino Tinto Crianza', 13.5),
(10, 2, 'Vino Blanco Gran Reserva', 12.5),
(11, 1, 'Vino Rosado Crianza', 12),
(12, 2, 'Vino Tinto Reserva', 14),
(13, 1, 'Vino Blanco Joven Especial', 11.5),
(14, 2, 'Vino Tinto Joven Especial', 13);

-- Insertar datos adicionales en Jóvenes
INSERT INTO Jovenes (vinoId, tiempoBarrica, tiempoBotella) VALUES
(5, 0, 6),
(7, 0, 12),
(13, 3, 9),
(14, 3, 6);

-- Insertar datos adicionales en Crianzas
INSERT INTO Crianzas (vinoId, tiempoBarrica, tiempoBotella) VALUES
(6, 6, 18),
(8, 6, 18),
(9, 6, 18),
(10, 12, 12),
(11, 12, 12),
(12, 12, 12);

-- Insertar datos adicionales en Cosechas para vinos de crianza
INSERT INTO Cosechas (cosechaId, vinoId, año, calidad) VALUES
(4, 6, 2021, 'Excelente'),
(5, 6, 2020, 'Muy buena'),
(6, 6, 2019, 'Buena'),
(7, 8, 2021, 'Excelente'),
(8, 8, 2020, 'Muy buena'),
(9, 8, 2019, 'Buena'),
(10, 9, 2021, 'Excelente'),
(11, 9, 2020, 'Muy buena'),
(12, 9, 2019, 'Buena'),
(13, 9, 2018, 'Regular'),
(14, 10, 2021, 'Excelente'),
(15, 10, 2020, 'Muy buena'),
(16, 10, 2019, 'Buena'),
(17, 10, 2018, 'Regular'),
(18, 11, 2021, 'Excelente'),
(19, 11, 2020, 'Muy buena'),
(20, 11, 2019, 'Buena'),
(21, 12, 2021, 'Excelente'),
(22, 12, 2020, 'Muy buena'),
(23, 12, 2019, 'Buena');

-- Insertar datos adicionales en Uvas
INSERT INTO Uvas (uvaId, nombre) VALUES
(4, 'Cabernet Sauvignon'),
(5, 'Merlot'),
(6, 'Syrah');
```

```
  -- Insertar datos adicionales en UvasVinos para vinos de crianza
  INSERT INTO VinosUvas (vinoId, uvaId) VALUES
  (6, 1),
  (6, 2),
  (8, 3),
  (8, 4),
  (9, 5),
  (9, 1),
  (9, 2),
  (10, 3),
  (10, 4),
  (11, 5),
  (11, 1),
  (12, 2),
  (12, 3);

  -- Insertar datos adicionales en UvasVinos para vinos jóvenes
  INSERT INTO VinosUvas (vinoId, uvaId) VALUES
  (5, 1),
  (5, 2),
  (7, 3),
  (7, 4),
  (13, 5),
  (13, 1),
  (14, 2);
*/
END;
//
delimiter ;

CALL p_populate_db();
```

18. Consultas

Bodegas/queries.sql

```
--
-- Autor: David Ruiz
-- Fecha: Octubre de 2024
-- Descripción: Consultas para implemtar ejemplos de álgebra relacional
--

-- Seleccionar todas las bodegas con denominación de origen Rioja:
SELECT *
FROM bodegas
WHERE denominacion_origen = 'Rioja'
;

-- Listado de vinos con sus uvas:
SELECT v.nombre, u.nombre
FROM vinos v
  JOIN vinos_uvas vu ON v.vino_id = vu.vino_id
  JOIN uvas u ON vu.uva_id = u.uva_id
;

-- Vinos con sus cosechas (crianza):
```

```sql
SELECT *
  FROM vinos v
  JOIN cosechas c ON v.vino_id = c.vino_id
;

-- Mostrar todas las bodegas que producen vinos tanto jóvenes como crianzas:
SELECT DISTINCT b.nombre
  FROM bodegas b
  JOIN vinos v ON b.bodega_id = v.bodega_id
;

-- Nombre de las bodegas y vinos que están compuestos, al menos, con uva
"Tempranillo":
SELECT b.nombre, v.nombre
FROM bodegas b
  JOIN vinos v ON b.bodega_id = v.bodega_id
  JOIN vinos_uvas vu ON v.vino_id = vu.vino_id
  JOIN uvas u ON vu.uva_id = u.uva_id
WHERE u.nombre = 'Tempranillo'
;

-- Total de cosechas por crianza:
SELECT c.vino_id, COUNT(*)
FROM cosechas c
  JOIN crianzas cr ON c.vino_id = cr.vino_id
GROUP BY c.vino_id
;

-- Nombre del vino joven con más grados:
SELECT v.nombre, MAX(v.grados)
  FROM vinos v
    JOIN jovenes j ON v.vino_id  = j.vino_id
;

-- Contar cuántas cosechas "Excelente" ha tenido cada vino:
SELECT v.nombre, COUNT(*)
FROM vinos v
  JOIN cosechas c ON v.vino_id = c.vino_id
  -- WHERE c.calidad = 'Excelente'
GROUP BY v.nombre
;
```

19. Modelo Tecnológico (v2)

Modelo tecnológico. Versión 2: Una relación para cada subclase

20. Script SQL para crear la base de datos

Bodegas2/createDB.sql

```sql
--
-- Autor: David Ruiz
-- Fecha: Octubre de 2024
-- Descripción: Script para crear la BD del ejercicio de Bodegas, con una
-- tabla para cada subclase
--
```

```sql
DROP DATABASE IF exists Bodegas2DB;
CREATE DATABASE Bodegas2DB;
USE Bodegas2DB;

DELIMITER //
CREATE OR REPLACE PROCEDURE p_create_db()
BEGIN
  CREATE OR REPLACE TABLE bodegas (
    bodega_id INT NOT NULL AUTO_INCREMENT,
    nombre VARCHAR(255) NOT NULL UNIQUE,
    denominacion_origen VARCHAR(255) NOT NULL,
    PRIMARY KEY (bodega_id),
    CONSTRAINT RN_1_Unicidad UNIQUE (nombre)
  );

  CREATE OR REPLACE TABLE jovenes (
    joven_id VARCHAR(32),
    bodega_id INT NOT NULL,
    nombre VARCHAR(255) NOT NULL,
    grados DECIMAL(5, 2) NOT NULL,
    tiempo_barrica INT NOT NULL,
    tiempo_botella INT NOT NULL,
    CONSTRAINT RN_2_TiempoCrianza
      CHECK (tiempo_barrica <=6 AND tiempo_barrica + tiempo_botella <=12 ),
    CONSTRAINT RN_3_Grados
      CHECK (grados >= 10 AND grados <= 15),
    PRIMARY KEY (joven_id),
    FOREIGN KEY (bodega_id) REFERENCES bodegas(bodega_id),
    CONSTRAINT RN_1_Unicidad UNIQUE (nombre)
  );

  CREATE OR REPLACE TABLE crianzas (
    crianza_id VARCHAR(32),
    bodega_id INT NOT NULL,
    nombre VARCHAR(255) NOT NULL,
    grados DECIMAL(5, 2) NOT NULL,
    tiempo_barrica INT NOT NULL,
    tiempo_botella INT NOT NULL,
    CONSTRAINT RN_2_TiempoCrianza
      CHECK (tiempo_barrica >=6 AND tiempo_barrica + tiempo_botella >= 24 ),
    CONSTRAINT RN_3_Grados
      CHECK (grados >= 10 AND grados <= 15),
    PRIMARY KEY (crianza_id),
    FOREIGN KEY (bodega_id) REFERENCES bodegas(bodega_id),
    CONSTRAINT RN_1_Unicidad UNIQUE(nombre)
  );

  CREATE OR REPLACE TABLE uvas (
    uva_id INT AUTO_INCREMENT,
    nombre VARCHAR(255) NOT NULL,
    PRIMARY KEY (uva_id),
    CONSTRAINT RN_1_Unicidad UNIQUE (nombre)
  );

  CREATE OR REPLACE TABLE cosechas (
```

```
    cosecha_id INT AUTO_INCREMENT,
    crianza_id VARCHAR(32) NOT NULL,
    año YEAR NOT NULL,
    calidad VARCHAR(255) NOT NULL,
    PRIMARY KEY (cosecha_id),
    FOREIGN KEY (crianza_id) REFERENCES crianzas(crianza_id)
      ON DELETE CASCADE,
    CONSTRAINT RN_4_CosechasAño UNIQUE (cosecha_id, año)
  );

  CREATE OR REPLACE TABLE vinos_uvas(
    vino_uva_id INT AUTO_INCREMENT,
    joven_id VARCHAR(32),
    crianza_id VARCHAR(32),
    uva_id INT NOT NULL,
    UNIQUE (joven_id, uva_id),
    UNIQUE (crianza_id, uva_id),
    PRIMARY KEY (vino_uva_id),
    FOREIGN KEY (joven_id) REFERENCES jovenes(joven_id),
    FOREIGN KEY (crianza_id) REFERENCES crianzas(crianza_id),
    FOREIGN KEY (uva_id) REFERENCES uvas(uva_id)
  );

END;
//
DELIMITER ;

CALL p_create_db();
```

21. Script SQL para la carga inicial de datos

Bodegas2/populateDB.sql

```
--
-- Autor: David Ruiz
-- Fecha: Octubre de 2024
-- Descripción: Script para poblar la BD del ejercicio de Bodegas
--

delimiter //
CREATE OR REPLACE PROCEDURE p_populate_db()
BEGIN
  -- Vaciar tablas
  DELETE FROM vinos_uvas;
  DELETE FROM cosechas;
  DELETE FROM jovenes;
  DELETE FROM crianzas;
  DELETE FROM uvas;
  DELETE FROM bodegas;
  -- Insertar datos en Bodegas
  INSERT INTO bodegas (bodega_id, nombre, denominacion_origen) VALUES
    (1, 'Bodegas El Sol', 'Rioja'),
    (2, 'Bodegas La Luna', 'Ribera del Duero')
  ;

  -- Insertar datos en Vinos
  INSERT INTO jovenes (joven_id, bodega_id, nombre, grados, tiempo_barrica,
```

```sql
tiempo_botella) VALUES
    ('j1', 1, 'Vino Blanco Joven', 12, 0, 6),
    ('j2', 2, 'Vino Tinto Joven', 13, 0, 12)
  ;

  -- Insertar datos en Crianzas
  INSERT INTO crianzas (crianza_id, bodega_id, nombre, grados, tiempo_barrica,
tiempo_botella)
  VALUES
    ('c1', 1, 'Vino Crianza Especial', 14, 6, 18),
    ('c2', 2, 'Vino Crianza Reserva', 13.5, 12, 12)
  ;

  -- Insertar datos en Uvas
  INSERT INTO uvas (uva_id, nombre) VALUES
    (1, 'Tempranillo'),
    (2, 'Garnacha'),
    (3, 'Albarino')
  ;

  -- Insertar datos en Cosechas
  INSERT INTO cosechas (cosecha_id, crianza_id, año, calidad) VALUES
    (1, 'c1', 2020, 'Excelente'),
    (2, 'c1', 2019, 'Buena'),
    (3, 'c2', 2018, 'Muy buena')
  ;

  -- Insertar datos en VinosUvas
  INSERT INTO vinos_uvas(joven_id, crianza_id, uva_id) VALUES
    ('j1', NULL, 3),
    ('j2', NULL, 1),
    (NULL, 'c1', 2),
    (NULL, 'c1', 1),
    (NULL, 'c2', 2),
    (NULL, 'c2', 1)
  ;

  -- Insertar datos adicionales en Vinos
  INSERT INTO jovenes (joven_id, bodega_id, nombre, grados, tiempo_barrica,
tiempo_botella) VALUES
    ('j3', 1, 'Vino Rosado Joven', 11.5, 0, 6),
    ('j4', 1, 'Vino Blanco Joven Especial', 11.5, 0, 12),
    ('j5', 2, 'Vino Tinto Joven Especial', 13, 3, 9)
  ;

  INSERT INTO crianzas (crianza_id, bodega_id, nombre, grados, tiempo_barrica,
tiempo_botella) VALUES
    ('c3', 1, 'Vino Blanco Reserva', 12.5, 6, 18),
    ('c4', 2, 'Vino Tinto Gran Reserva', 14, 6, 18),
    ('c5', 2, 'Vino Blanco Crianza', 13, 6, 18),
    ('c6', 1, 'Vino Tinto Crianza', 13.5, 12, 12),
    ('c7', 2, 'Vino Blanco Gran Reserva', 12.5, 12, 12),
    ('c8', 1, 'Vino Rosado Crianza', 12, 12, 12),
    ('c9', 2, 'Vino Tinto Reserva', 14, 12, 12)
  ;
```

```
  -- Insertar datos adicionales en Cosechas para vinos de crianza
  INSERT INTO cosechas (cosecha_id, crianza_id, año, calidad) VALUES
    (4, 'c3', 2021, 'Excelente'),
    (5, 'c3', 2020, 'Muy buena'),
    (6, 'c3', 2019, 'Buena'),
    (7, 'c4', 2021, 'Excelente'),
    (8, 'c4', 2020, 'Muy buena'),
    (9, 'c4', 2019, 'Buena'),
    (10, 'c5', 2021, 'Excelente'),
    (11, 'c5', 2020, 'Muy buena'),
    (12, 'c5', 2019, 'Buena'),
    (13, 'c5', 2018, 'Regular'),
    (14, 'c6', 2021, 'Excelente'),
    (15, 'c6', 2020, 'Muy buena'),
    (16, 'c6', 2019, 'Buena'),
    (17, 'c6', 2018, 'Regular'),
    (18, 'c7', 2021, 'Excelente'),
    (19, 'c7', 2020, 'Muy buena'),
    (20, 'c7', 2019, 'Buena'),
    (21, 'c8', 2021, 'Excelente'),
    (22, 'c8', 2020, 'Muy buena'),
    (23, 'c8', 2019, 'Buena')
  ;

  -- Insertar datos adicionales en Uvas
  INSERT INTO uvas (uva_id, nombre) VALUES
    (4, 'Cabernet Sauvignon'),
    (5, 'Merlot'),
    (6, 'Syrah')
  ;

  -- Insertar datos adicionales en UvasVinos para vinos de crianza
  INSERT INTO vinos_uvas (joven_id, crianza_id, uva_id) VALUES
    (NULL, 'c3', 1),
    (NULL, 'c3', 2),
    (NULL, 'c4', 3),
    (NULL, 'c4', 4),
    (NULL, 'c5', 5),
    (NULL, 'c5', 1),
    (NULL, 'c5', 2),
    (NULL, 'c6', 3),
    (NULL, 'c6', 4),
    (NULL, 'c7', 5),
    (NULL, 'c7', 1),
    (NULL, 'c8', 2),
    (NULL, 'c8', 3),
    ('j3', NULL, 1),
    ('j3', NULL, 2),
    ('j4', NULL, 3),
    ('j4', NULL, 4),
    ('j5', NULL, 5),
    ('j5', NULL, 1)
  ;
END;
//
delimiter ;
```

```
CALL p_populate_db();
```

22. Consultas

Bodegas2/queries.sql

```sql
--
-- Autor: David Ruiz
-- Fecha: Octubre de 2024
-- Descripción: Consultas para implemtar ejemplos de álgebra relacional
--

-- Seleccionar todas las bodegas con denominación de origen Rioja:
SELECT *
FROM bodegas
WHERE denominacion_origen = 'Rioja'
;

-- Vista auxiliar con todos los vinos
CREATE OR REPLACE VIEW v_vinos AS
  SELECT j.joven_id vino_id, j.bodega_id, j.nombre, j.grados
  FROM jovenes j
  UNION
  SELECT c.crianza_id vino_id, c.bodega_id, c.nombre, c.grados
  FROM crianzas c
;

-- Listado de vinos con sus uvas:
SELECT v.nombre, u.nombre
FROM v_vinos v JOIN vinos_uvas vu
  ON  (v.vino_id = vu.joven_id OR v.vino_id = vu.crianza_id) JOIN uvas u
  ON u.uva_id = vu.uva_id
ORDER BY v.nombre
;

-- Vinos con sus cosechas (crianza):
SELECT *
FROM crianzas c JOIN cosechas co
  ON c.crianza_id = co.crianza_id
;

-- Mostrar todas las bodegas que producen vinos tanto jóvenes como crianzas:
SELECT DISTINCT b.nombre
FROM bodegas b  JOIN v_vinos v
  ON b.bodega_id = v.bodega_id
;

-- Nombre de las bodegas y vinos que están compuestos, al menos, con uva
"Tempranillo":
SELECT v.vino_id, b.nombre nombre_bodega, v.nombre nombre_vino
FROM bodegas b JOIN v_vinos v
  ON  b.bodega_id=v.bodega_id JOIN vinos_uvas vu
  ON (v.vino_id = vu.joven_id OR v.vino_id = vu.crianza_id) JOIN uvas u
  ON vu.uva_id = u.uva_id
WHERE
  u.nombre = 'Tempranillo'
```

```
ORDER by b.nombre
;

-- Total de cosechas pr crianza:
SELECT c.crianza_id, COUNT(*)
FROM cosechas c JOIN crianzas cr
  ON c.crianza_id = cr.crianza_id
GROUP BY c.crianza_id
;

-- Nombre del vino joven con más grados:
SELECT j.nombre, MAX(j.grados)
  FROM jovenes j
;

-- Contar cuántas cosechas "Excelente" ha tenido cada vino:
SELECT c.crianza_id, c.nombre, COUNT(*)
FROM crianzas c JOIN cosechas co
  ON c.crianza_id = co.crianza_id
WHERE co.calidad = 'Excelente'
GROUP BY c.nombre
;
```

23. SQL Avanzado

Realice un disparador que compruebe que en la tabla VinosUvas no se insertan tuplas con valores para jovenId y crianzaId simultáneamente:

Bodegas2/triggers.sql

```
--
-- Autor: David Ruiz
-- Fecha: Octubre de 2024
-- Descripción: Comprobar que las FK de VInosUvas son disjuntas
--

DELIMITER //
CREATE OR REPLACE PROCEDURE p_check_id_disjuntos
  (id1 VARCHAR(32), id2 VARCHAR(32))
BEGIN
  IF (id1 = id2) OR (id1 = '' AND id2 = '') OR (id1 <> '' AND id2 <> '') THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Las FK de VinosUvas tienen que ser
disjuntas';
  END if;
END;
//
DELIMITER ;

DELIMITER //
CREATE or REPLACE TRIGGER t_bi_vinos_uvas
BEFORE INSERT ON vinos_uvas FOR EACH ROW
BEGIN
  call p_check_id_disjuntos(NEW.joven_id, NEW.crianza_id);
END
//
DELIMITER ;
```

```
DELIMITER //
CREATE or REPLACE TRIGGER t_bu_vinos_uvas
BEFORE UPDATE ON vinos_uvas FOR EACH ROW
BEGIN
  call p_check_id_disjuntos(NEW.joven_id, NEW.crianza_id);
END
//
DELIMITER ;
```

24. Pruebas SQL

Pruebas SQL **Bodegas2/tests.sql**

```
--
-- Autor: David Ruiz
-- Fecha: Octubre de 2024
-- Descripción: Pruebas de aceptación en SQL para Bodegas y Vinos
--

-- Inicialización de la BD con estado consistente
USE Bodegas2DB;

-- TABLA DE RESULTADOS DE TESTS
CREATE OR REPLACE TABLE test_results (
  test_id VARCHAR(10) NOT NULL PRIMARY KEY,
  test_name VARCHAR(200) NOT NULL,
  test_message VARCHAR(500) NOT NULL,
  test_status ENUM('PASS', 'FAIL', 'ERROR') NOT NULL,
  execution_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- PROCEDIMIENTO AUXILIAR PARA LOGGING
DELIMITER //
CREATE OR REPLACE PROCEDURE p_log_test(
  IN p_test_id VARCHAR(10),
  IN p_message VARCHAR(500),
  IN p_status ENUM('PASS', 'FAIL', 'ERROR')
)
BEGIN
  INSERT INTO test_results(test_id, test_name, test_message, test_status)
  VALUES (p_test_id, SUBSTRING_INDEX(p_message, ':', 1), p_message, p_status);
END //
DELIMITER ;

-- ====================
-- TESTS DE BODEGAS
-- ====================

-- 1. Crear una nueva bodega con todos los datos correctos según las reglas de
negocio
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_bodega_1()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('B.1', 'bodegas: Crear bodega válida', 'FAIL');
```

```
  CALL p_populate_db();
  INSERT INTO bodegas(nombre, denominacion_origen) VALUES ('Bodega la Nueva',
'Rias Baixas');
  CALL p_log_test('B.1', 'bodegas: Crear bodega válida', 'PASS');
END //
DELIMITER ;

-- 2. Crear una nueva bodega sin nombre.
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_bodega_2()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('B.2', 'bodegas: nombre NOT NULL', 'PASS');

  CALL p_populate_db();
  INSERT INTO bodegas(nombre, denominacion_origen) VALUES (NULL, 'Rias Baixas');
  CALL p_log_test('B.2', 'ERROR: nombre NULL permitido', 'FAIL');
END //
DELIMITER ;

-- 3. Crear una nueva bodega con el nombre repetido.
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_bodega_3()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('B.3', 'bodegas: nombre UNIQUE', 'PASS');

  CALL p_populate_db();
  INSERT INTO bodegas(nombre, denominacion_origen) VALUES ('Bodega la Nueva',
'Rias Baixas');
  INSERT INTO bodegas(nombre, denominacion_origen) VALUES ('Bodega la Nueva',
'Rias Baixas');
  CALL p_log_test('B.3', 'ERROR: nombre duplicado permitido', 'FAIL');
END //
DELIMITER ;

-- 4. Crear una nueva bodega sin denominación de origen.
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_bodega_4()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('B.4', 'bodegas: denominacion_origen NOT NULL', 'PASS');

  CALL p_populate_db();
  INSERT INTO bodegas(nombre, denominacion_origen) VALUES ('Bodega la Nueva',
NULL);
  CALL p_log_test('B.4', 'ERROR: denominacion_origen NULL permitido', 'FAIL');
END //
DELIMITER ;

-- ====================
-- TESTS DE VINOS
-- ====================

-- 1. Crear un nuevo vino Joven con todos los datos correctos según las reglas de
```

```
negocio
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_vino_1()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('V.1', 'jovenes: Crear joven válido', 'FAIL');

  CALL p_populate_db();
  INSERT INTO jovenes (joven_id, bodega_id, nombre, grados, tiempo_barrica,
tiempo_botella)
  VALUES ('j10', 1, 'Vino nuevo Joven', 11, 0, 12);
  CALL p_log_test('V.1', 'jovenes: Crear joven válido', 'PASS');
END//
DELIMITER ;

-- 2. Crear un nuevo vino Crianza con todos los datos correctos según las reglas
de negocio
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_vino_2()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('V.2', 'crianzas: Crear crianza válida', 'FAIL');

  CALL p_populate_db();
  INSERT INTO crianzas (crianza_id, bodega_id, nombre, grados, tiempo_barrica,
tiempo_botella)
  VALUES ('c10', 1, 'Vino nuevo Crianza', 11, 12, 12);
  CALL p_log_test('V.2', 'crianzas: Crear crianza válida', 'PASS');
END//
DELIMITER ;

-- 3. Crear un nuevo vino Joven sin nombre.
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_vino_3()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('V.3', 'jovenes: nombre NOT NULL', 'PASS');

  CALL p_populate_db();
  INSERT INTO jovenes (joven_id, bodega_id, nombre, grados, tiempo_barrica,
tiempo_botella)
  VALUES ('j10', 1, NULL, 11, 0, 12);
  CALL p_log_test('V.3', 'ERROR: nombre NULL permitido en jovenes', 'FAIL');
END//
DELIMITER ;

-- 4. Crear un nuevo vino Crianza sin nombre.
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_vino_4()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('V.4', 'crianzas: nombre NOT NULL', 'PASS');

  CALL p_populate_db();
  INSERT INTO crianzas (crianza_id, bodega_id, nombre, grados, tiempo_barrica,
tiempo_botella)
```

```
  VALUES ('c10', 1, NULL, 11, 12, 12);
  CALL p_log_test('V.4', 'ERROR: nombre NULL permitido en crianzas', 'FAIL');
END//
DELIMITER ;

-- 5. Crear un nuevo vino Joven con el nombre repetido.
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_vino_5()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('V.5', 'jovenes: nombre UNIQUE', 'PASS');

  CALL p_populate_db();
  INSERT INTO jovenes (joven_id, bodega_id, nombre, grados, tiempo_barrica,
tiempo_botella)
  VALUES ('j10', 1, 'Vino nuevo Joven', 11, 0, 12);
  INSERT INTO jovenes (joven_id, bodega_id, nombre, grados, tiempo_barrica,
tiempo_botella)
  VALUES ('j11', 1, 'Vino nuevo Joven', 11, 0, 12);
  CALL p_log_test('V.5', 'ERROR: nombre duplicado permitido en jovenes', 'FAIL');
END//
DELIMITER ;

-- 6. Crear un nuevo vino Crianza con el nombre repetido.
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_vino_6()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('V.6', 'crianzas: nombre UNIQUE', 'PASS');

  CALL p_populate_db();
  INSERT INTO crianzas (crianza_id, bodega_id, nombre, grados, tiempo_barrica,
tiempo_botella)
  VALUES ('c10', 1, 'Vino nuevo Crianza', 11, 12, 12);
  INSERT INTO crianzas (crianza_id, bodega_id, nombre, grados, tiempo_barrica,
tiempo_botella)
  VALUES ('c11', 1, 'Vino nuevo Crianza', 11, 12, 12);
  CALL p_log_test('V.6', 'ERROR: nombre duplicado permitido en crianzas', 'FAIL');
END//
DELIMITER ;

-- 7. Crear un nuevo vino Joven sin grados de alcohol.
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_vino_7()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('V.7', 'jovenes: grados NOT NULL', 'PASS');

  CALL p_populate_db();
  INSERT INTO jovenes (joven_id, bodega_id, nombre, grados, tiempo_barrica,
tiempo_botella)
  VALUES ('j10', 1, 'Vino nuevo Joven', NULL, 0, 12);
  CALL p_log_test('V.7', 'ERROR: grados NULL permitido en jovenes', 'FAIL');
END//
DELIMITER ;
```

```
-- 8. Crear un nuevo vino Crianza sin grados de alcohol.
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_vino_8()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('V.8', 'crianzas: grados NOT NULL', 'PASS');

  CALL p_populate_db();
  INSERT INTO crianzas (crianza_id, bodega_id, nombre, grados, tiempo_barrica,
tiempo_botella)
  VALUES ('c10', 1, 'Vino nuevo Crianza', NULL, 12, 12);
  CALL p_log_test('V.8', 'ERROR: grados NULL permitido en crianzas', 'FAIL');
END//
DELIMITER ;

-- 9. Crear un nuevo vino Joven con graduación incorrecta.
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_vino_9()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('V.9', 'jovenes: grados CHECK (10-15)', 'PASS');

  CALL p_populate_db();
  INSERT INTO jovenes (joven_id, bodega_id, nombre, grados, tiempo_barrica,
tiempo_botella)
  VALUES ('j10', 1, 'Vino nuevo Joven', 9, 0, 12);
  INSERT INTO jovenes (joven_id, bodega_id, nombre, grados, tiempo_barrica,
tiempo_botella)
  VALUES ('j11', 1, 'Vino nuevo Joven', 16, 0, 12);
  CALL p_log_test('V.9', 'ERROR: grados fuera de rango permitido en jovenes',
'FAIL');
END//
DELIMITER ;

-- 10. Crear un nuevo vino Crianza con graduación incorrecta.
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_vino_10()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('V.10', 'crianzas: grados CHECK (10-15)', 'PASS');

  CALL p_populate_db();
  INSERT INTO crianzas (crianza_id, bodega_id, nombre, grados, tiempo_barrica,
tiempo_botella)
  VALUES ('c10', 1, 'Vino nuevo Crianza', 9, 12, 12);
  INSERT INTO crianzas (crianza_id, bodega_id, nombre, grados, tiempo_barrica,
tiempo_botella)
  VALUES ('c11', 1, 'Vino nuevo Crianza', 16, 12, 12);
  CALL p_log_test('V.10', 'ERROR: grados fuera de rango permitido en crianzas',
'FAIL');
END//
DELIMITER ;

-- 11. Crear un nuevo vino Joven sin bodega.
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_vino_11()
```

```
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('V.11', 'jovenes: bodega_id NOT NULL', 'PASS');

  CALL p_populate_db();
  INSERT INTO jovenes (joven_id, bodega_id, nombre, grados, tiempo_barrica,
tiempo_botella)
  VALUES ('j10', NULL, 'Vino nuevo Joven', 10, 0, 12);
  CALL p_log_test('V.11', 'ERROR: bodega_id NULL permitido en jovenes', 'FAIL');
END//
DELIMITER ;

-- 12. Crear un nuevo vino Crianza sin bodega.
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_vino_12()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('V.12', 'crianzas: bodega_id NOT NULL', 'PASS');

  CALL p_populate_db();
  INSERT INTO crianzas (crianza_id, bodega_id, nombre, grados, tiempo_barrica,
tiempo_botella)
  VALUES ('c10', NULL, 'Vino nuevo Crianza', 11, 12, 12);
  CALL p_log_test('V.12', 'ERROR: bodega_id NULL permitido en crianzas', 'FAIL');
END//
DELIMITER ;

-- 2. Crear un nuevo vino Crianza con todos los datos correctos según las reglas
de negocio
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_vino_2()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('V.2', 'crianzas: Crear crianza válida', 'FAIL');

  CALL p_populate_db();
  CALL p_insert_crianza('c10', 1, 'Vino nuevo Crianza', 11, 12, 12);
  CALL p_log_test('V.2', 'crianzas: Crear crianza válida', 'PASS');
END//
DELIMITER ;

-- 3. Crear un nuevo vino Joven sin nombre.
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_vino_3()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('V.3', 'jovenes: nombre NOT NULL', 'PASS');

  CALL p_populate_db();
  CALL p_insert_joven('j10', 1, null, 11, 0, 12);
  CALL p_log_test('V.3', 'ERROR: nombre NULL permitido en jovenes', 'FAIL');
END//
DELIMITER ;

-- 4. Crear un nuevo vino Crianza sin nombre.
DELIMITER //
```

```
CREATE OR REPLACE PROCEDURE p_test_vino_4()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('V.4', 'crianzas: nombre NOT NULL', 'PASS');

  CALL p_populate_db();
  CALL p_insert_crianza('c10', 1, null, 11, 12, 12);
  CALL p_log_test('V.4', 'ERROR: nombre NULL permitido en crianzas', 'FAIL');
END//
DELIMITER ;

-- 5. Crear un nuevo vino Joven con el nombre repetido.
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_vino_5()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('V.5', 'jovenes: nombre UNIQUE', 'PASS');

  CALL p_populate_db();
  CALL p_insert_joven('j10', 1, 'Vino nuevo Joven', 11, 0, 12);
  CALL p_insert_joven('j11', 1, 'Vino nuevo Joven', 11, 0, 12);
  CALL p_log_test('V.5', 'ERROR: nombre duplicado permitido en jovenes', 'FAIL');
END//
DELIMITER ;

-- 6. Crear un nuevo vino Crianza con el nombre repetido.
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_vino_6()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('V.6', 'crianzas: nombre UNIQUE', 'PASS');

  CALL p_populate_db();
  CALL p_insert_crianza('c10', 1, 'Vino nuevo Crianza', 11, 12, 12);
  CALL p_insert_crianza('c11', 1, 'Vino nuevo Crianza', 11, 12, 12);
  CALL p_log_test('V.6', 'ERROR: nombre duplicado permitido en crianzas', 'FAIL');
END//
DELIMITER ;

-- 7. Crear un nuevo vino Joven sin grados de alcohol.
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_vino_7()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('V.7', 'jovenes: grados NOT NULL', 'PASS');

  CALL p_populate_db();
  CALL p_insert_joven('j10', 1, 'Vino nuevo Joven', null, 0, 12);
  CALL p_log_test('V.7', 'ERROR: grados NULL permitido en jovenes', 'FAIL');
END//
DELIMITER ;

-- 8. Crear un nuevo vino Crianza sin grados de alcohol.
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_vino_8()
BEGIN
```

```
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('V.8', 'crianzas: grados NOT NULL', 'PASS');

  CALL p_populate_db();
  CALL p_insert_crianza('c10', 1, 'Vino nuevo Crianza', null, 12, 12);
  CALL p_log_test('V.8', 'ERROR: grados NULL permitido en crianzas', 'FAIL');
END//
DELIMITER ;

-- 9. Crear un nuevo vino Joven con graduación incorrecta.
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_vino_9()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('V.9', 'jovenes: grados CHECK (10-15)', 'PASS');

  CALL p_populate_db();
  CALL p_insert_joven('j10', 1, 'Vino nuevo Joven', 9, 0, 12);
  CALL p_insert_joven('j11', 1, 'Vino nuevo Joven', 16, 0, 12);
  CALL p_log_test('V.9', 'ERROR: grados fuera de rango permitido en jovenes',
'FAIL');
END//
DELIMITER ;

-- 10. Crear un nuevo vino Crianza con graduación incorrecta.
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_vino_10()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('V.10', 'crianzas: grados CHECK (10-15)', 'PASS');

  CALL p_populate_db();
  CALL p_insert_crianza('c10', 1, 'Vino nuevo Crianza', 9, 12, 12);
  CALL p_insert_crianza('c11', 1, 'Vino nuevo Crianza', 16, 12, 12);
  CALL p_log_test('V.10', 'ERROR: grados fuera de rango permitido en crianzas',
'FAIL');
END//
DELIMITER ;

-- 11. Crear un nuevo vino Joven sin bodega.
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_vino_11()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('V.11', 'jovenes: bodega_id NOT NULL', 'PASS');

  CALL p_populate_db();
  CALL p_insert_joven('j10', null, 'Vino nuevo Joven', 10, 0, 12);
  CALL p_log_test('V.11', 'ERROR: bodega_id NULL permitido en jovenes', 'FAIL');
END//
DELIMITER ;

-- 12. Crear un nuevo vino Crianza sin bodega.
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_vino_12()
BEGIN
```

```
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('V.12', 'crianzas: bodega_id NOT NULL', 'PASS');

  CALL p_populate_db();
  CALL p_insert_crianza('c10', null, 'Vino nuevo Crianza', 11, 12, 12);
  CALL p_log_test('V.12', 'ERROR: bodega_id NULL permitido en crianzas', 'FAIL');
END//
DELIMITER ;

-- ====================
-- EJECUTAR TODOS LOS TESTS
-- ====================

DELIMITER //
CREATE OR REPLACE PROCEDURE p_run_all_tests()
BEGIN
  -- Limpiar resultados anteriores
  DELETE FROM test_results;

  -- Tests de Bodegas
  CALL p_test_bodega_1();
  CALL p_test_bodega_2();
  CALL p_test_bodega_3();
  CALL p_test_bodega_4();

  -- Tests de Vinos
  CALL p_test_vino_1();
  CALL p_test_vino_2();
  CALL p_test_vino_3();
  CALL p_test_vino_4();
  CALL p_test_vino_5();
  CALL p_test_vino_6();
  CALL p_test_vino_7();
  CALL p_test_vino_8();
  CALL p_test_vino_9();
  CALL p_test_vino_10();
  CALL p_test_vino_11();
  CALL p_test_vino_12();

  -- Mostrar resultados
  SELECT * FROM test_results ORDER BY test_id;

  -- Resumen
  SELECT
    test_status,
    COUNT(*) as count
  FROM test_results
  GROUP BY test_status;
END //
DELIMITER ;

-- Ejecutar todos los tests
CALL p_run_all_tests();
```

25. Pruebas HTTP

Pruebas HTTP